

MutPy

Lenguaje: Python

Forma de utilizarse:

```
import unittest
from calculator import add, subtract, multiply, divide

class TestCalculator(unittest.TestCase):

    def test_add(self):
        self.assertEqual(add(2, 3), 5)
        self.assertEqual(add(-1, 1), 0)

    def test_subtract(self):
        self.assertEqual(subtract(5, 3), 2)
        self.assertEqual(subtract(0, 4), -4)

    def test_multiply(self):
        self.assertEqual(multiply(4, 3), 12)
        self.assertEqual(multiply(-2, 3), -6)

    def test_divide(self):
        self.assertEqual(divide(6, 3), 2)
        with self.assertRaises(ValueError):
            divide(5, 0)

if __name__ == '__main__':
    unittest.main()
```

```
[*] Start mutation process:
- targets: calculator.py
- tests: test_calculator.py
[*] 4 tests passed:
- test_calculator [0.00000 s]
[*] Start mutants generation and execution:
- [# 1] AOR calculator: [0.00100 s] killed by test_add (test_calculator.TestCalculator)
- [# 2] AOR calculator: [0.00000 s] killed by test_subtract (test_calculator.TestCalculator)
- [# 3] AOR calculator: [0.00100 s] killed by test_multiply (test_calculator.TestCalculator)
- [# 4] AOR calculator: [0.00100 s] killed by test_multiply (test_calculator.TestCalculator)
- [# 5] AOR calculator: [0.00100 s] killed by test_multiply (test_calculator.TestCalculator)
- [# 6] AOR calculator: [0.00000 s] survived
- [# 7] AOR calculator: [0.00000 s] killed by test_divide (test_calculator.TestCalculator)
- [# 8] COI calculator: [0.00100 s] killed by test_divide (test_calculator.TestCalculator)
- [# 9] ROR calculator: [0.00103 s] killed by test_divide (test_calculator.TestCalculator)
[*] Mutation score [0.04790 s]: 88.9%
- all: 9
- killed: 8 (88.9%)
- survived: 1 (11.1%)
- incompetent: 0 (0.0%)
- timeout: 0 (0.0%)
```

Donde utilizar esta herramienta:

1. Aplicaciones web

Pruebas de mutación en proyectos web para garantizar que la lógica de negocio y las rutas de tu aplicación estén correctamente cubiertas.

- Ejemplo:
Proyecto: Una API REST desarrollada con Flask que maneja autenticación, creación de usuarios y operaciones CRUD.
- Uso:
Pruebas de mutación para verificar que los controladores (endpoints) no pasen mutaciones no detectadas, como saltar validaciones o cambiar condiciones de retorno.

2. Bibliotecas de análisis de datos

Proyectos donde se desarrollan funciones matemáticas, estadísticas o de procesamiento de datos, para asegurarse de que las pruebas detectan errores en cálculos.

- Ejemplo:
Proyecto: Una función que calcula promedios ponderados y estadísticas básicas.
- Uso:
Verificar que errores en fórmulas matemáticas sean detectados por las pruebas.

3. Algoritmos de Machine Learning

Pruebas en proyectos que implementan algoritmos desde cero o manipulan datos antes de alimentar a modelos.

- Ejemplo:
Proyecto: Implementación personalizada de K-Nearest Neighbors (KNN).
 - Uso:
Probar que las pruebas cubren correctamente posibles mutaciones en funciones críticas como la distancia euclidiana o la selección de vecinos más cercanos.
4. *Aplicaciones con lógica compleja*
Proyectos con lógica de negocio compleja, como reglas de cálculo o decisiones basadas en múltiples condiciones.
- Ejemplo:
Proyecto: Un software de nómina que calcula salarios y deducciones.
 - Uso:
Verificar que cambios en las fórmulas o reglas sean detectados por las pruebas unitarias.
5. *Sistemas de simulación*
Proyectos donde se modelan sistemas o procesos con múltiples interacciones.
- Ejemplo:
Proyecto: Un simulador de colas para un supermercado (como el que estás desarrollando).
 - Uso:
Probar que las pruebas cubren errores en las condiciones de asignación de clientes a cajeros.